Student’s name and surname: Krzysztof Nazar
ID: 184698
Cycle of studies: master’s degree studies
Mode of study: full-time studies
Field of study: Informatics
Specialization: Distributed Applications and Internet Systems

MASTER'S THESIS

Title of the thesis: Utilization of named entity recognition classes in entity linking

Title of the thesis (in Polish): Wykorzystanie klas nazw własnych w procesie linkowania encji

Supervisor: dr inż. Mariusz Matuszek

STRESZCZENIE

Łączenie Encji (ang. *Entity Linking, EL*) to zadanie polegające na dopasowywaniu nazw własnych w tekście do odpowiadających im wpisów w bazie wiedzy. Niniejsza praca prezentuje system Łączenia Encji, który łączy rozpoznawanie nazw własnych (ang. *Named Entity Recognition, NER*) z rozstrzyganiem nazw własnych (ang. *Named Entity Disambiguation, NED*) przy użyciu bazy wiedzy DBpedia. Tekst wejściowy jest przetwarzany przez jeden z trzech modeli NER, z których każdy wykorzustuje inną listę typów encji. Celem tego przetwarzania jest uzyskanie rozkładu prawdopodobieństwa przynależności każdej nazwy własnej do poszczególnych klas. Następnie z bazy wiedzy DBpedia pobierani są kandydaci do połączenia z wykrytymi encjami. Każdy kandydat zostaje oceniony za pomocą czterech metryk kontekstowych oraz trzech metryk opartych na wektorach osadzenia typów, które porównują najbardziej prawdopodobne typy NER nazwy własnej z typami ontologicznymi kandydata pobranego z DBpedii. Zestaw uzyskanych metryk dla każdego kandydata dla danej nazwy własnej służy jako cechy wejściowe do modelu XGBoost, uczącego się wybierać najlepiej dopasowanego kandydata do nazwy własnej i otaczającego kontekstu. Badania eksperymentalne przeprowadzone na zbiorach testowych AIDA i ACE2004 wykazują, że uwzględnienie typów NER w procesie rozstrzygania nazw własnych pozwala osiągnąć dokładność linkowania kandydatów na poziomie około 86% - 90%. Połączenie wyników NER, semantycznego dopasowania za pomocą osadzeń oraz modelu uczenia maszynowego czyni zaproponowane podejście szybkim, przejrzystym i semantycznie bogatym rozwiązaniem, które można zastosować w celu dodania wiarygodnych informacji w systemach odpowiadających na pytania, wyszukiwania informacji oraz rozbudowy grafów wiedzy.

Słowa kluczowe: Łączenie Encji, Rozpoznawanie Nazw Własnych, Rozróżnianie Nazw Własnych, DBpedia

Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki inżynieryjne i techniczne, inżynieria informatyczna

ABSTRACT

Entity Linking (EL) is the task of matching mentions of entities in text to their corresponding entries in a knowledge base. This thesis develops an end-to-end Entity Linking system that combines fine-grained Named Entity Recognition (NER) with context-aware Named Entity Disambiguation (NED) over the DBpedia knowledge base. First, input text is processed by one of three NER models—each offering a different level of class granularity—to produce a probability distribution over entity types for every mention. Next, candidate entities are retrieved from DBpedia. Each candidate is then evaluated by four contextual scorers and three novel type-embedding scorers that compare the mention's most probable NER-types with each candidate's ontology-types. These scores serve as features to an XGBoost ranker, which learns to select the best-matching candidate from the list of candidates fetched from DBpedia. Experimental evaluation on the AIDA and ACE2004 benchmarks shows that incorporating NER class distributions into the disambiguation process allows for linking accuracy of approximately 86% - 90%. By uniting probabilistic NER-type outputs, embedding-based semantic matching, and a learned ranking model, the proposed framework delivers a fast, interpretable, and semantically rich EL solution suitable for downstream tasks such as question answering, information retrieval, and knowledge graph population, with the main goal of extending the context by relevant information retrieved from the knowledge base.

Keywords: Entity Linking, Named Entity Recognition, Named Entity Disambiguation, DBpedia

LIST OF IMPORTANT SYMBOLS AND ABBREVIATIONS

- **NLP** - Natural Language Processing
- **NE** - Named Entity
- **NEL** - Named Entity Linking
- **NER** - Named Entity Recognition
- **NED** - Named Entity Disambiguation
- **KB** - Knowledge Base
- **API** - Application Programming Interface
- **URI** - Unique Resource Identifier
- **RDF** - Resource Description Framework

Contents

1. INTRODUCTION

This chapter outlines the primary objectives and the motivation behind developing the system presented in this thesis. The source code for the proposed solution is available on GitHub and it is distributed under the GPL-3.0 license, ensuring it is accessible as an open-source project for other developers and researchers[1].

1.1 Objective of the Thesis

The primary objective of this thesis is to design and implement a custom Named Entity Linking (NEL) system that leverages class information assigned during the Named Entity Recognition (NER) phase to enhance the Named Entity Disambiguation (NED) process. The system is intended to be a practical, efficient, and accessible solution that can be easily deployed without any extensive configuration.

Entity Linking (EL) plays a fundamental role in Natural Language Processing (NLP). Its main goal is to identify named entities in unstructured text and link them to corresponding entries in a structured Knowledge Base (KB). This process facilitates a deeper semantic understanding of textual data. The proposed solution integrates both NER and NED components, each addressing distinct but complementary challenges within the EL pipeline.

The NER component uses one of the three selected fine-tuned NER models. This model can detect and classify named entities while assigning the probability distributions over multiple fine-grained predefined classes, which are specific to each type of model. This probabilistic representation aims to capture the inherent ambiguity of entity types more effectively than single-label classification approaches.

The NED component addresses the subsequent task of resolving ambiguities when a recognized entity mention could refer to multiple candidate entities in the DBpedia Knowledge Base. To improve disambiguation accuracy, this thesis investigates a combination of techniques including context-aware matching, similarity-based scoring, and the use of probabilistic class information from the NER model. The integration of these methods aims to make the decision making process more reliable and transparent.

By addressing these challenges in both NER and NED, this work contributes to the broader development of effective EL systems, with applications extending to related areas such as Wikification and Named Entity Linking[36]. The effectiveness of the proposed methods has been evaluated and discussed in detail in Section 7 of this thesis.

[1] `https://github.com/Danzigerrr/ProbNEL`

2. PROBLEM STATEMENT

In the following section introduces the problem of Named Entity Linking together with its core components and challenges. This section provides an in-depth look at the steps involved in the NEL process: Named Entity Recognition, which identifies and classifies entity mentions in the input text, and Named Entity Disambiguation, which resolves these recognized mentions to their correct entities in the knowledge base.

2.1 Named Entity Linking

Named Entity Linking is a fundamental task in Natural Language Processing[35]. It involves identifying entities mentioned in text and linking them to their corresponding entries in a KB[34]. Two widely used examples of knowledge bases are DBpedia[3] and Wikidata[40]. This task is crucial for enabling machines to understand the semantic meaning of text and retrieve contextual knowledge effectively. For instance, given a mention "Paris" in a sentence, it could refer to Paris, the capital of France, or Paris, a character in Greek mythology. NEL ensures that such mentions are resolved to the correct records in the KB. This is how NEL can be used to enrich text with new information using the structured data stored in the KB[6].

Named Entity Linking is a specific part of the broader Entity Linking task that concentrates exclusively on connecting Named Entities (NE) to a knowledge base, deliberately excluding links to common phrases. By limiting the linking process to named entities, NEL reduces ambiguity and makes it easier to create reliable gold standards[22]. However, this restriction also means that some valuable information may be lost, especially when the outputs of the entity linking system are used for further analysis. In the system presented in this thesis, the focus will be on named entities only; therefore, common phrases will be omitted.

The process of entity linking can be divided into two main steps[31]:

- **Named Entity Recognition (NER):** Identifying mentions of named entities in unstructured text.
- **Named Entity Disambiguation (NED):** Linking identified mentions to their corresponding unique entities in a knowledge base.

However, some newer methods solve the whole process in one step using a joint entity linking approach. A major problem with the two-step method is that mistakes made during NER carry over to NED, with no way to fix them later. Because of this, recent research has developed methods that do both tasks together to reduce errors and improve linking accuracy[39]. On the other hand, building such joint systems is generally more complex, requiring greater computational resources and often involving substantially advanced machine learning techniques such as reinforcement learning[13].

For this project, the two-step approach will be used. To explain this clearly, each step will be described in a greater detail in the following sections.

2.1.1 Challenges associated with Named Entity Linking

The problem of NEL is challenging due to the ambiguity of language, the dynamic nature of knowledge bases, and the diversity of contexts in which named entities might appear. Ambiguity arises when a single mention in the text can refer to multiple records in the KB, requiring the system to infer the correct interpretation based on often subtle contextual clues. Moreover, the dynamic nature of KBs further complicates NEL, as they are constantly updated with new entities and evolving descriptions, necessitating systems that can adapt and incorporate changes efficiently. Additionally, named entities are referenced in diverse contexts, from formal documents to colloquial social media posts, each presenting unique linguistic challenges such as idiomatic expressions, or abbreviations. These factors collectively demand reliable approaches capable of understanding both textual context and the underlying structure of the KB.

2.1.2 Practical Applications of Entity Linking

Entity Linking systems have broad applications, including:

- **Information Extraction:** Entity linking resolves ambiguity in extracted entities and relations, improving the quality of relation extraction[34, 21].
- **Information Retrieval:** Disambiguated entities enhance semantic search by enabling more accurate understanding of web document content[34, 8].
- **Content Analysis:** Entity linking supports topical classification, which improves applications such as content-based news recommendation systems[25, 27].
- **Question Answering:** Accurate disambiguation of entity mentions enables knowledge-based systems to generate precise answers[43, 44].
- **Knowledge Base Population:** Entity linking facilitates the automatic enrichment of knowledge bases with newly extracted and correctly linked information[20].

2.2 Named Entity Recognition

Named Entity Recognition (NER) focuses on identifying spans of text that correspond to named entities. [15] This process involves not only detecting named entities in the text, but also assigning a part of speech tag/class to each detected entity. These tags could involve: persons, organizations, locations, dates, and other predefined categories. For example, in the sentence "Paris is known for the Eiffel Tower, designed by Gustave Eiffel.", NER identifies "Paris" as a location, "Eiffel Tower" as a Monument of Landmark, and "Gustave Eiffel" as a person. This example is presented in Figure 2.1. It is important to note, that each of the recognized NE is linked to an entry in a knowledge base - each with a Unique Resource Identifier (URI).

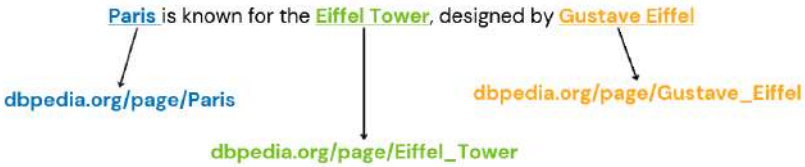

Figure 2.1. Named Entity Recognition example

2.2.1 Approaches in Solving the Problem

Named Entity Recognition (NER) has progressed through several key approaches, including:

- **Rule-Based Systems:** These were the earliest solutions for NER, relying on handcrafted rules and predefined lists of entities, known as gazetteers. Rules might include identifying capitalised words as potential entities or recognising names that follow titles like "Dr." or "Mr.". Although effective in constrained settings, these systems struggle with unseen entities and language variations. For instance, they might correctly identify "Dr. Brown" as a person but fail to recognise unconventional names like "Elon Musk" unless explicitly listed. These methods often use regular expressions combined with terminological resources, but their main disadvantage is the manual construction of rules, which is time-consuming and domain-dependent[12].

- **Machine Learning Models:** These improved NER by introducing supervised learning techniques such as Conditional Random Fields (CRFs) and Support Vector Machines (SVMs). These models learn patterns from labelled datasets using features like word positions and part-of-speech tags. Unlike rule-based systems, they generalise better to varied data. For example, a CRF model trained on news articles could correctly classify "Barack Obama" as a person and "Silicon Valley" as a location, drawing on patterns learned from similar training examples[29].

- **Deep Learning Approaches:** These revolutionised NER by capturing context and semantics through neural architectures. Models such as BiLSTM-CRF combine bidirectional LSTMs for context representation with CRFs for sequence labelling, achieving strong results. Transformer-based models like BERT further advanced NER with pre-trained contextual embeddings, enabling nuanced understanding. For example, a BERT-based system can distinguish between "Apple launched a new product" (organisation) and "I ate an apple" (fruit) based on context[19].

- **Multi-Label NER:** This approach allows entities to belong to multiple categories, addressing cases where roles overlap. For instance, a "concert" could be classified as both an "event" and a "work of art". Multi-label models achieve this by using techniques like sigmoid activations, assigning probabilities to multiple classes and providing richer representations of entities. Such fine-grained entity recognition often involves multi-class, multi-label classification frameworks that separate entity segmentation and classification[23].

2.2.2 Challenges associated with Named Entity Recognition

The challenges associated with NER include:

- **Ambiguity in entity types:** it occurs when a single mention can belong to multiple categories depending on the context. For instance, the word "Apple" might refer to the company Apple Inc., or to the fruit. Accurately interpreting such cases requires a nuanced understanding of the surrounding context.
- **Language Variability:** entities are often expressed in diverse ways, including abbreviations, synonyms, or even misspellings. For instance, "U.S." and "United States" both refer to the same entity but are written differently, posing a challenge for less robust models to treat them as equivalent.
- **Domain-Specific Entities:** many NER systems are trained on general datasets, which limits their ability to recognise specialised terms or names from specific fields such as medicine, law, or technology. Without additional domain-specific training or data augmentation, these models often fail to capture such entities.
- **Complex Contexts:** can obscure entities, especially in cases where entities appear in intricate sentence structures, are part of idiomatic expressions, or are referred to implicitly via pronouns. For example, resolving references like "He was born there" requires connecting pronouns to their antecedents based on prior context, which is a non-trivial task for NER systems. These challenges collectively demand sophisticated techniques and models capable of adapting to a wide range of linguistic variations and contexts.

2.3 Named Entity Disambiguation

The Named Entity Disambiguation process addresses the challenge of determining the correct entity from a knowledge base for a given mention in text. Firstly, a set of possible entries has to be generated from KB. Next, the most suitable entry has to be chosen and finally linked to the detected named entity from the text.

2.3.1 Named Entity Disambiguation Process

The Named Entity Disambiguation process typically involves two main steps: Candidate Generation and Candidate Disambiguation.

Candidate Generation

In this stage, potential matches (also called "candidates") for a mention in the text are identified from the KB. These candidates represent all possible entities that a mention could correspond to, based on its surface form or variations of it. For instance, in the sentence "Paris is known for the Eiffel Tower

and the Louvre Museum." the mention "Paris" could refer to several entities, as presented in Figure 2.2.

Figure 2.2. Named Entity Disambiguation example - candidate generation

Efficient candidate generation relies heavily on precomputed mappings between mentions and entities, often stored in inverted indices or lookup tables. For example, a mapping might list that "Paris" is linked to URIs corresponding to different interpretations. Moreover, techniques like fuzzy matching can help identify candidates even if the mention is misspelled or abbreviated. For instance, "Páris" or "Paris, TX" should still map to appropriate candidates.

Candidate Disambiguation

After generating candidates, the next step is to rank and select the most appropriate entity based on the context of the mention. This involves several strategies:

- **Context Matching:** This method compares the surrounding text of the mention with information associated with each candidate in the KB. For instance, if "Paris" appears in a sentence about European capitals, the context strongly suggests Paris (France) rather than Paris, Texas.

- **Similarity Metrics:** Semantic similarity metrics, such as Cosine Similarity between word embeddings or TF-IDF vectors, are commonly used. For example, if "Paris" is in a discussion involving the Eiffel Tower, the similarity between these terms' embeddings would favour Paris (France).

- **Graph-Based Approaches:** These leverage the structure of the KB by considering relationships between entities. For example, if the text mentions "Barack Obama" and "Hawaii", a graph-based model might strengthen the link to Barack Obama based on a "born in" relationship between the two entities.

Candidate disambiguation may also incorporate additional features of KB data, such as popularity (e.g., more widely known entities might be ranked higher) or recency (e.g., events or entities relevant to current affairs).

Eventually, in order to create a meaningful comparison, some kind of ranking strategy has to be designed. Thus, each candidate can be for example assigned multiple scores, what allows for easy sorting and selection of the most suitable candidate. An example of candidate disambiguation and candidate ranking is shown in Figure 2.3.

Figure 2.3. Named Entity Disambiguation example - candidate disambiguation based on score

2.3.2 Challenges Associated with Named Entity Disambiguation

The challenges associated with NED include:

- **Entity Ambiguity:** Many entities have multiple potential references in a knowledge base. For instance, the entity "Jaguar" could refer to a car brand, an animal, or a software platform.
- **Knowledge Base Limitations:** Knowledge bases may lack coverage for newly emerging entities or domain-specific entities.
- **Noisy Input Data:** Errors from upstream NER tasks or misspellings in the input text can complicate the disambiguation process.

3. RELATED WORK

This section provides an overview of key research developments relevant to this thesis. It examines prior work on entity linking, focusing on approaches that both include and exclude Named Entity Recognition class information in the Entity Linking pipeline. By examining the strengths and limitations of existing methods, this chapter identifies knowledge gap that motivates the integration of fine-grained NER outputs into entity disambiguation process.

3.1 Entity Linking Without NER Class Integration

Traditional entity linking systems often treated Named Entity Recognition as a separate preprocessing step or ignored type information altogether. The AIDA system pioneered graph-based disambiguation by leveraging coherence among entities but relied primarily on hard-coded entity popularity scores rather than semantic type matching [17]. Similarly, BLINK established a strong neural baseline using BERT embeddings for candidate ranking but processed mentions independently without exploiting predicted entity classes [42]. DBpedia Spotlight, another widely used tool, detects mentions and links them to DBpedia resources by generating candidates via the DBpedia Lookup API and applying statistical disambiguation models[26]. While it can filter candidates by DBpedia ontology classes, it does not explicitly use traditional NER type labels or type distributions in its linking process. While these approaches achieved competitive performance, they left significant potential untapped in aligning semantic types between mentions and knowledge base entries.

3.2 Type-Aware Entity Linking Approaches

Recent research increasingly highlights the importance of incorporating type information into entity linking. NER4EL demonstrated that fine-grained NER outputs can improve candidate generation through type-based filtering, especially in low-resource scenarios [37]. DeepType introduced a neural-symbolic framework that learns optimal type systems to constrain candidate selection to compatible ontology classes [30]. Another modern approach enhanced entity embeddings with hierarchical type information from WordNet, improving disambiguation accuracy across multiple languages [18].

Building on these advances, ReFinED presents an efficient end-to-end entity linking model that jointly performs mention detection, fine-grained entity typing, and disambiguation in a single forward pass[4]. Its transformer-based architecture integrates Wikipedia-derived type information as soft semantic signals. By leveraging fine-grained types alongside entity descriptions, ReFinED achieves state-of-the-art accuracy while being over 60 times faster than comparable models[5].

Despite these advancements, many type-aware systems still rely on hard type constraints that may discard valid candidates and fail to capture the full value of NER outputs.

3.3 Filling the Knowledge Gap

This thesis addresses two key limitations in current type-aware entity linking systems. First, the underutilization of NER confidence distributions: most systems rely solely on the top predicted class instead of leveraging the full probability distribution over entity types. Second, shallow type compatibility checks, where type matching is often based on simple equality or coarse embeddings rather than semantic similarity. The growing availability of fine-grained NER systems-such as Few-NERD's 66-class taxonomy [11]-offers an opportunity to improve entity linking with detailed type signals. The presented approach introduces three novel type-embedding scorers that compare SBERT-encoded NER class distributions with DBpedia ontology types, enabling richer semantic matching. This extends DeepType's neural-symbolic philosophy [30] by overcoming its rigid type constraints and enhances NER4EL's filtering [37] through soft semantic matching rather than binary decisions. Moreover, the presented method aligns with findings that type-enhanced embeddings improve EL accuracy without sacrificing efficiency [18], and addresses observations by Ganea et al. (2017) that purely neural EL methods often struggle with explicit type reasoning [14].

4. NER IMPLEMENTATION

This section describes the implementation of the Named Entity Recognition (NER) step, including the justification for selection of NER models used in the system, particularly in relation to the subsequent Named Entity Disambiguation (NED) task.

4.1 Defining a Set of Named Entity Types

A fundamental step in NER is the definition of a comprehensive and well-structured set of entity types. This categorization is crucial because it directly influences the NER system's ability to accurately identify and classify named entities, which in turn is vital for the downstream NED process. Each class should have a clear definition, which can be used for example during the annotation process. The definitions of NER classes used in the datasets described in the following sections are based on the original publications in which these three datasets were introduced.

Three different NER models were used in NER implementation:

- Model trained on the CoNLL++ dataset[1]
- Model trained on the OntoNotes 5.0 dataset[2]
- Model trained on the Few-NERD dataset[3]

These three models were selected because they represent different levels of entity type granularity, allowing to analyse how the detail and variety of entity classes affect the performance of Named Entity Disambiguation. A diverse set of entity types might be helpful during the NED process. In this project the top predicted entity NER types and their associated confidence scores assigned by NER model will be used to select the best possible matching candidate within the candidates fetched from a knowledge base. This type-based matching might be particularly important for resolving ambiguities that arise when different entities share the same name but belong to distinct categories. For instance, the name "Washington" could refer to the capital city, the U.S. state, or the historical figure George Washington. There is a high chance, that by leveraging the fine-grained entity types provided by a model like the Few-NERD model and establishing mappings between these NER types and the ontology type systems of knowledge graphs, presented end-to-end entity linking approach might more accurately determine the matching candidate.

[1]Dataset: CoNLL++ dataset, NER model: tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context
[2]Dataset: OntoNotes 5.0 dataset, NER model: tomaarsen/span-marker-roberta-large-ontonotes5
[3]Dataset: Few-NERD dataset, NER model: tomaarsen/span-marker-roberta-large-fewnerd-fine-super

4.1.1 CoNLL-03 Dataset

One of the most influential early NER datasets is the CoNLL-03 shared task dataset [33], which introduced a foundational set of entity types that have since become standard in many NER systems. The dataset categorizes named entities into four coarse-grained classes, as summarized in Table 4.1.

Table 4.1. CoNLL-03 Named Entity Types and Descriptions

Entity Class Name	**Description**
PER (Persons)	Names of individuals, including fictional characters.
ORG (Organizations)	Names of companies, agencies, institutions, etc.
LOC (Locations)	Names of geographical locations such as countries, cities, and states.
MISC (Miscellaneous)	Names of miscellaneous entities that do not belong to the other groups.

While the PER, ORG, and LOC categories are relatively well defined, the MISC (Miscellaneous) category is notably broad and underspecified. It serves as a quite broad and ambiguous class that encompasses various types of entities that cannot be clearly assigned to the other three categories. This ambiguity can introduce challenges during both annotation and model training, potentially reducing classification precision.

To address known annotation issues in the original test set, an enhanced version of the dataset, referred to as CoNLL+, was later introduced[4]. This version corrects specific annotation errors in the test split while preserving the original training and development data.

The CoNLL++ dataset includes the following components:

- `/data/conllpp_train.txt` and `/data/conllpp_dev.txt`: The original training and development sets from the CoNLL-03 dataset, sourced from Named-Entity-Recognition-NER-Papers.
- `/data/conllpp_test.txt`: A manually corrected test set containing 186 sentences that differ from the original CoNLL-03 test set.

The CoNLL-03 dataset, along with its improved variant CoNLL++, provides a widely used and standardized benchmark for evaluating NER systems. Despite its limited class set, it remains relevant due to its historical significance and continued use in comparative research.

4.1.2 OntoNotes 5.0 Dataset

The OntoNotes 5.0 dataset [41] represents a major advancement in named entity annotation, introducing a significantly richer and more fine-grained taxonomy. Unlike the coarse-grained CoNLL-03 schema, OntoNotes 5.0 defines 18 distinct entity types, enabling more nuanced semantic interpretation of text. This makes it particularly suitable for downstream tasks, where detailed class information can aid in disambiguation. The complete list of OntoNotes 5.0 entity types, along with their descriptions, is provided in Table 4.2.

[4] `https://github.com/ZihanWangKi/CrossWeigh`

Table 4.2. OntoNotes 5.0 Named Entity Types and Descriptions

Entity Class Name	Description
PERSON	People, including fictional
NORP	Nationalities or religious or political groups
FACILITY	Buildings, airports, highways, bridges, etc.
ORGANIZATION	Companies, agencies, institutions, etc.
GPE	Countries, cities, states
LOCATION	Non-GPE locations, mountain ranges, bodies of water
PRODUCT	Vehicles, weapons, foods, etc. (Not services)
EVENT	Named hurricanes, battles, wars, sports events, etc.
WORK OF ART	Titles of books, songs, etc.
LAW	Named documents made into laws
LANGUAGE	Any named language
DATE	Absolute or relative dates or periods
TIME	Times smaller than a day
PERCENT	Percentage (including "%")
MONEY	Monetary values, including unit
QUANTITY	Measurements, as of weight or distance
ORDINAL	"first", "second"
CARDINAL	Numerals that do not fall under another type

4.1.3 Few-NERD Dataset

The Few-NERD dataset[11] represents a significant step forward by defining a rich hierarchy of 8 coarse-grained and 66 fine-grained entity types, covering a much broader spectrum of categories. The hierarchical structure allows for both broad and specific entity identification. The coarse-grained categories provide a high-level understanding, while the fine-grained types offer the nuanced distinctions essential for effective NED.

While the official Few-NERD paper does not provide explicit definitions for each of the 66 fine-grained entity types, it illustrates their usage through examples. Figure 4.1 showcases examples of named entities tagged with various fine-grained location types within the Few-NERD dataset.

Coarse Type	Fine Type	Example
Location	GPE	The company moved to a new office in *Las Vegas*, *Nevada*.
	Body of Water	The *Finke River* normally drains into the Simpson Desert to the north west of the Macumba.
	Island	An invading army of Teutonic Knights conquered *Gotland* in 1398.
	Mountain	C.G.E. Mannerheim met Thubten Gyatso in *Wutai Shan* during the course of his expedition from Turkestan to Peking.
	Park	*Victoria Park* contains examples of work by several architects including Alfred Waterhouse (Xaverian College).
	Road/Transit	The thirty-first race of the 1951 season was held on October 7 at the one-mile dirt *Occoneechee Speedway*.
	Other	Herodotus (7.59) reports that *Doriscus* was the first place Xerxes the Great stopped to review his troops.

Figure 4.1. Examples of Few-NERD Fine-Grained Location Entity Types. Source: page 14 in [11]

The hierarchy of 8 coarse-grained and 66 fine-grained entity types is shown in Figure 4.2

Figure 4.2. Few-NERD Coarse-Grained Entity Classes. Source:
https://production-media.paperswithcode.com/datasets/few-nerd.png

The extensive and fine-grained typology of Few-NERD enables NER systems to go beyond identifying general entity categories and distinguish between entities that might share the same surface form but have different underlying meanings. This capability might be advantageous for NED, where resolving such ambiguities is a core challenge. The comprehensive and nuanced classification system of Few-NERD provides a robust foundation for training models that can perform well, especially in scenarios requiring the handling of rare or previously unseen entity types, such as few-shot learning. The detailed entity typing offered by Few-NERD is therefore crucial for achieving more accurate and context-aware disambiguation in subsequent NED tasks.

4.2 NER Models Used in the System

This project employs and compares several pre-trained Named Entity Recognition models within the complete Entity Linking process. To ensure a meaningful comparison, each model utilizes a different set of entity classes. The identified named entities will be assigned a set of NER classes and their associated probabilities will be used during the subsequent Named Entity Disambiguation step. The primary objective of this comparison is to determine which NER model offers the most valuable output, in terms of identified and classified named entities, for the NED process.

4.2.1 Selected Models

The selected models are state-of-the-art NER models trained using SpanMarker[1]. SpanMarker is a framework built on the Hugging Face Transformers library, inheriting features like efficient training

and hyperparameter tuning. SpanMarker offers a range of models that are accessible for anyone via HuggingFace[5].

CoNLL++ Model

The model `span-marker-xlm-roberta-large-conllpp-doc-context`[6] is based on the xlm-roberta-large encoder[9] and was trained on the CoNLL++ dataset. As stated before, CoNLL++ is an improved version of the original CoNLL-03 dataset, incorporating some corrections into the test dataset. This NER model leverages document-level context during training and achieves a competitive F1 score of 95.5.

Ontonotes 5.0 Model

The model `span-marker-roberta-large-ontonotes5`[7] is based on RoBERTa large model[24]. This NER model was trained on the OntoNotes 5.0 dataset, achieving an F1 score of 91.54. For comparison, a strong spaCy model (`en_core_web_trf`)[8] reaches an F1 score of 89.8. This SpanMarker model utilizes a roberta-large encoder[9].

To better align with the evaluation requirements of this project, certain NER types from the OntoNotes 5.0 dataset were excluded. Specifically, entity types like "cardinal" and "ordinal" are not considered crucial for evaluating the end-to-end entity linking performance. Furthermore, attempting to link cardinal or ordinal entities to a knowledge graph is often unproductive, as knowledge bases typically do not contain distinct entities for such numerical or sequential identifiers. As indicated on page 22 of the OntoNotes 5.0 documentation[10], these types represent useful textual components but are not typically proper nouns that would correspond to entities in a knowledge graph. Therefore, these non-proper noun entity types were disregarded for the purposes of this evaluation.

Few-NERD Model

The model `span-marker-bert-base-fewnerd-fine-super`[11] is based on BERT base model (cased)[10]. This NER model was trained on the fine-grained, supervised Few-NERD dataset. It achieved a Test F1 score of 70.53, which is competitive within the all-time leaderboard for Few-NERD models using a bert-base encoder[12]. The model identifies and classifies named entities into 66 fine-grained entity types.

[5] https://huggingface.co/models?library=span-marker
[6] https://huggingface.co/tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context
[7] https://huggingface.co/tomaarsen/span-marker-roberta-large-ontonotes5
[8] https://spacy.io/models/en#en_core_web_trf
[9] https://huggingface.co/FacebookAI/roberta-large
[10] https://catalog.ldc.upenn.edu/docs/LDC2013T19/OntoNotes-Release-5.0.pdf
[11] https://huggingface.co/tomaarsen/span-marker-bert-base-fewnerd-fine-super
[12] https://paperswithcode.com/sota/few-shot-ner-on-few-nerd-inter

5. NED IMPLEMENTATION

This section details the Named Entity Disambiguation (NED) process employed in the system, which utilizes the DBpedia knowledge base. The following section describes steps involved in retrieving candidate entities from DBpedia, as well as the methods for disambiguating these candidates and ranking them to identify the most appropriate link for a given named entity mention.

5.1 DBpedia Knowledge Base

DBpedia is a collaboratively created, multilingual knowledge graph extracted from the content of Wikipedia. It aims to structure the information found in the infoboxes within Wikipedia articles, making this information available in a machine-readable format. As detailed on its website[1], DBpedia comprises tens of millions of resources representing real-world entities across diverse domains such as people, places, organizations, events, and creative works.

The knowledge graph is largely built through automated processes that parse structured data from Wikipedia infoboxes, as explained in the DBpedia documentation[2]. This extracted information is then modelled using the Resource Description Framework (RDF). The project also benefits from community contributions that help in refining the data and expanding the DBpedia ontology. An overview of the available datasets and their characteristics can be found on the DBpedia datasets page[3].

5.1.1 Ontology

The DBpedia ontology is fundamental to organizing the vast information within the knowledge graph. As outlined in the ontology classes documentation[4], classes in the DBpedia ontology serve as categories that define the types of entities. These classes are structured hierarchically, with `owl:Thing` acting as the root class, similar to `entity` in Wikidata. All other DBpedia ontology classes are subclasses, either directly or indirectly, of this root.

For instance, common high-level classes in the DBpedia ontology include `dbo:Person`, representing individuals; `dbo:Place`, encompassing geographical locations; and `dbo:Event`, categorizing occurrences. The prefix `dbo:` conventionally denotes classes within the core DBpedia ontology.

Entities within DBpedia are associated with these ontology classes through the RDF property `rdf:type`[5], which indicates the class membership of an entity. A prime example is the entity of Albert Einstein[6], which has `rdf:type dbo:Person`[7], classifying him as a person in the DBpedia ontology.

[1] `https://www.dbpedia.org/`
[2] `https://wiki.dbpedia.org/`
[3] `https://wiki.dbpedia.org/develop/datasets`
[4] `https://mappings.dbpedia.org/server/ontology/classes/`
[5] `https://www.w3.org/TR/rdf11-concepts/#dfn-rdf-type`
[6] `https://dbpedia.org/page/Albert_Einstein`
[7] `http://dbpedia.org/ontology/Person`

Other typess assinged to this entity are: `dbo:Scientist`[8] and `dbo:Agent`[9]. This typing mechanism not only categorizes entities but also enables inheritance of properties from parent classes, facilitating structured querying and retrieval of information based on entity types, a crucial aspect for effective candidate selection and disambiguation in the designed NED pipeline.

This assignment of types is not merely a categorization mechanism; it also enables hierarchical reasoning. As subclasses inherit properties and characteristics from their superclasses, the `rdf:type` connection between entities and ontology classes becomes a powerful tool for organizing and, more importantly, for retrieving structured information from DBpedia. By querying based on these types, one can efficiently access entities belonging to specific categories or their subclasses, which is particularly useful during the candidate selection and disambiguation phases of the NED process.

5.1.2 Retrieving candidates from DBpedia

Two primary methodologies exist for retrieving data from DBpedia:

1. **DBpedia SPARQL Endpoint**[10]: This method permits the submission of SPARQL queries to a server, facilitating data retrieval in various formats. It necessitates the construction of a SPARQL query adhering to the correct syntax, followed by the extraction of the output. While offering considerable flexibility, this approach demands the creation of bespoke queries. Empirical observations suggest that it exhibits comparatively slower performance relative to the second method. Data can be returned in multiple formats, including JSON, XML, CSV, and others. Example result of the query using DBpedia SPARQL Endpoint can be accessed using this url.
2. **DBpedia Lookup**[11]: This method employs an API endpoint to retrieve a set of information concerning one or more entities from DBpedia. The request can incorporate supplementary parameters to refine the query. observations indicate that the DBpedia Lookup API generates responses significantly more rapidly. Conversely, it provides reduced flexibility due to the constrained and predefined set of parameters available for query formulation. What is more, The DBpedia Lookup Service can return search results in either XML or JSON. An example of DBpedia Lookup query result can be accessed using this url.

For the Named Entity Disambiguation process, the DBpedia Lookup API was selected as a more suitable method for retrieving candidate entities for this project. This decision is driven by the observed significant performance advantage in terms of response time compared to querying the SPARQL endpoint. While the SPARQL endpoint offers greater flexibility in creating complex queries, the speed at which the Lookup API returns relevant entity information is crucial for the efficiency of the real-time disambiguation pipeline. The slight reduction in query flexibility is an acceptable trade-off for the substantial gains in speed, ensuring a more responsive and scalable system. Moreover, since complex query capabilities are not required in this project, the DBpedia Lookup API presents a well-balanced and better-suiting choice for this project.

[8] `http://dbpedia.org/ontology/Scientist`
[9] `http://dbpedia.org/ontology/Agent`
[10] `https://dbpedia.org/sparql`
[11] `https://github.com/dbpedia/dbpedia-lookup`

5.2 Ranking Candidates Using Metrics

For each recognized and classified named entity identified during the NER process, a list of candidate entities is retrieved from the knowledge base by querying with the entity's surface form. To ensure that the Named Entity Disambiguation (NED) process remains balanced and capable of capturing various aspects of candidate relevance, a diverse set of metrics is introduced. These metrics evaluate different dimensions of similarity between the recognized entity and each of the retrieved candidates, supporting the identification of the most appropriate match. Each candidate is assigned a score by each metric. This resulting set of scores is then used to determine the most suitable candidate for the given named entity.

There are 4 contextual metrics designed and used in the system, namely:

- Levenshtein Distance
- Context Matching
- Popularity of the Candidate
- Position of the Candidate

Moreover, there are 3 metrics assessing the similarities between NER types assigned to the recognized named entity and the ontology types of a candidate, namely:

- Weighted Average of NER Type Embedding Similarities
- Maximum Similarity with Top-K NER Types
- Weighted Sum of Maximum NER Type Embedding Similarities

Each of these metrics is described in detail below, along with a justification for its use in the system.

5.2.1 Contextual Metrics

This section contains description of these metrics, along with a justification for its inclusion in the system. These scorers are applied separately, allowing flexibility in applying and tuning the scoring models during training and evaluation phases.

Levenshtein Distance (Surface Form Matching)

Levenshtein distance quantifies the similarity between two strings by measuring the minimum number of single-character edits (insertions, deletions, substitutions) required to change one string into the other. In the context of entity linking, this metric assesses the surface form similarity between the recognized entity label and the candidate's label or alias. A higher similarity score suggests that the candidate is linguistically close to the original entity mention, which often correlates with correct matches.

To compute this similarity, the `fuzzywuzzy` Python library[12] is used, specifically its `ratio` method. This method tokenizes the input strings and uses the underlying `difflib.ratio` function[13] to compute the edit distance between token sequences. The resulting score, which ranges from 0 to 100, is then normalized by dividing by 100 and rounded to three decimal places.

An illustrative example is provided in Listing 5.1, where the similarity score between the strings "The United States" and "The United States of America" is computed using the `fuzz.ratio` method.

```
from fuzzywuzzy import fuzz

score = fuzz.ratio("The United States", "The United States of America")
print(round(score/100,3))  # Output: 0.76
```

Listing 5.1: Levenshtein surface form distance calculation using `fuzz.ratio`

Context Matching (Similarity Between Sentence and Candidate Description Embeddings)

Context matching is a semantic similarity metric that compares the textual context surrounding a recognized entity with the description of each candidate entity retrieved from a knowledge base. Both the context and the candidate description are first transformed into dense vector representations—commonly referred to as embeddings—which capture the semantic meaning of the input texts.

To compute the similarity between these two embeddings, cosine similarity is used. This metric measures the cosine of the angle between two vectors in the embedding space, providing a score that reflects how similar the two texts are in meaning. Cosine similarity is particularly well-suited for high-dimensional semantic spaces, as it focuses on vector orientation rather than magnitude, making it robust to variations in sentence length or intensity.

For generating embeddings, the `Sentence-BERT (sBERT)` model[14] was employed. Unlike the original BERT architecture, which is optimized for token-level outputs, sBERT is specifically designed to produce semantically meaningful sentence-level embeddings that enable efficient and accurate similarity computation. This project uses the pretrained model `all-mpnet-base-v2`[15], known for its strong performance on semantic textual similarity tasks. The model maps the given text into a 768 dimensional dense vector space, that can be used for tasks like clustering or semantic search.

The final context matching score is derived by computing the cosine similarity between the context and candidate embeddings. The score is then normalized to the [0,1] interval and rounded to three decimal places.

An illustrative example is provided in Listing 5.2, where a short context sentence is compared with a candidate description using the same model and scoring function used in the implementation.

```
from sentence_transformers import SentenceTransformer, util

```

[12] `https://pypi.org/project/fuzzywuzzy/`
[13] SequenceMatcher ratio() function documentation
[14] `https://huggingface.co/sentence-transformers`
[15] `https://huggingface.co/sentence-transformers/all-mpnet-base-v2`

```
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

context = "Joe Biden, the former U.S. president, gave a speech at the White House."
candidate_description = "Joe Biden is the 46th president of the United States."

context_emb = model.encode(context, convert_to_tensor=True)
candidate_emb = model.encode(candidate_description, convert_to_tensor=True)

similarity_score = util.pytorch_cos_sim(context_emb, candidate_emb).item()
print(round(similarity_score, 3))  # Output: 0.691
```

Listing 5.2: Computing context similarity using sentence embeddings and cosine similarity

Popularity of the Candidate

Candidate popularity reflects the general prominence or visibility of an entity within the knowledge base. In the case of DBpedia, the `ref_count` property—indicating the number of other resources referencing the entity—is used as a proxy for its popularity.

To moderate the effect of extreme values, the raw reference counts are first transformed using the natural logarithm with an offset, via the $\log(1 + x)$ function. This transformation compresses the scale and better preserves relative differences among lower values. The resulting log values are then linearly normalized to the [0,1] interval, using the minimum and maximum values observed among all candidates for a given entity. This normalization ensures that popularity scores are comparable across different candidate sets and that no single candidate disproportionately dominates due to high absolute popularity.

If all candidates have the same reference count (or none at all), a default score of 1.0 is assigned to each, avoiding division by zero and ensuring neutral treatment.

An illustrative example is shown in Listing 5.3, where the popularity of three candidate entities is scored based on their `ref_count` values.

```
import numpy as np

def score_popularity(ref_count, min_log, max_log, round_to=3):
    log_val = np.log1p(ref_count)
    if max_log == min_log:
        return round(1.0, round_to)
    normalized = (log_val - min_log) / (max_log - min_log)
    return round(normalized, round_to)
```

```
9
10 ref_counts = [950, 125, 30]
11 log_counts = np.log1p(ref_counts)
12 min_log = np.min(log_counts)
13 max_log = np.max(log_counts)
14 scores = [score_popularity(rc, min_log, max_log) for rc in ref_counts]
15 print(scores)  # Output: [1.0, 0.41, 0.0]
```

Listing 5.3: Scoring candidate popularity (log-normalized reference counts)

Position of the Candidate

The order in which candidates are retrieved can provide a subtle yet informative information regarding their potential relevance. Usually, when a knowledge base is searched, the system ranks the candidates and returns those that match best. The system can search for best records using keywords, popularity, and other related terms[16]. Finally, a list of matching candidates is returned. Candidates appearing earlier in the list are more similar to the given query and are usually more popular, which makes them more likely to be the best possible match to the entity mention.

To leverage this implicit ranking, a straightforward scoring mechanism based on the candidate's position in the retrieval list is employed. Each candidate is assigned a raw position score corresponding to its index, starting from 1 for the first retrieved candidate. These raw position scores are then inverted and normalized to a $[0, 1]$ range. In this normalized scale, the candidate retrieved first receives the highest score of 1.0, with subsequent candidates receiving progressively lower scores. This normalization ensures that the positional score remains within a consistent and manageable range, preventing it from dominating other scoring metrics.

This method introduces a lightweight preference for candidates that the underlying knowledge base retrieval mechanism deems more relevant. Notably, initial experiments also considered directly utilizing the implicit DBpedia scores often associated with retrieved candidates. However, these scores exhibited significant variability in their magnitude, making consistent normalization and integration with other metrics challenging. Even the application of logarithmic normalization to the DBpedia scores did not yield satisfactory and stable results across different entity mentions. Therefore, the position-based scoring, due to its inherent normalization and ease of integration, was adopted as a more robust and manageable approach for capturing the implicit relevance signal from the candidate retrieval order.

```
1 def calculate_position_scores(num_candidates, round_to=3):
2     if num_candidates <= 0:
3         return []
4     raw_scores = list(range(1, num_candidates + 1))
5     if num_candidates == 1:
6         return [round(1.0, round_to)]
```

[16] https://github.com/dbpedia/lookup

```
    normalized_scores = [(num_candidates - (pos - 1)) / (num_candidates - 1) for pos in raw_scores]
    return [round(score, round_to) for score in normalized_scores]

num_candidates = 3
position_scores = calculate_position_scores(num_candidates)
print(position_scores)  # Output: [1.0, 0.5, 0.0]
```

Listing 5.4: Example of Position Score Calculation

5.2.2 NER-ontology types embeddings similarity metrics

Named Entity Disambiguation process leverages the output from the Named Entity Recognition process. When a NER model classifies a named entity, it provides a list of classes along with the probability of the entity belonging to each class. Observations indicate that the top-ranked classes often account for a significant portion of the total probability mass (e.g., upwards of 99% for the first three classes). Consequently, in the NED pipeline, these top predicted NER classes and their associated probabilities are used as the type information for each identified named entity.

Since this project employed multiple NER models, each with its own set of entity types, a flexible approach for comparing the predicted NER types with the ontology classes of candidate entities was designed. To address this requirement, an embedding-based method was developed to effectively measure the semantic similarity between these entity types and the ontology types of a candidate.

For each identified named entity, the top three predicted classes were selected based on their probability scores. The empirical analysis suggests that these top three classes generally capture approximately 99% of the total probability distribution over the predicted NER types. To enrich the semantic information associated with these NER classes, their descriptions were used, which are based on the original papers in which these NER datasets were described. The goal of adding these descriptions was to provide a more comprehensive semantic context for each NER class. For example, while the label *GPE* alone offers limited understanding, the expanded description *"GPE – Names of geographical administrative entities, including countries, villages, cities, states, provinces, prefectures, and other forms of municipalities"* provides a much clearer and more informative definition. The list of all created definitions can be found in Appendix with NER Types Definitions.

To generate the embeddings for these enriched NER class descriptions and the ontology classes of candidate entities, Sentence-BERT (SBERT)[32] were utilized. SBERT is a fine-tuned version of the BERT model, specifically engineered to produce high-quality sentence and short-text embeddings with improved efficiency. Unlike standard BERT, which processes sentences independently and requires computationally intensive pairwise comparisons for similarity assessment, SBERT employs a *siamese network architecture* to encode entire sentences into fixed-length vector representations. By applying *mean pooling* over the token embeddings, SBERT effectively captures the overall semantic meaning of a sentence, enabling fast and accurate similarity computations. This architecture significantly reduces the computational overhead and storage demands, making SBERT highly suitable for tasks like semantic

search, clustering, and textual similarity analysis[17]. In contrast to standard BERT, which lacks a direct mechanism for generating sentence-level embeddings and needs additional computationally expensive steps like cross-encoding, SBERT provides a scalable and performant solution for semantic similarity tasks.

Specifically, the version `all-MiniLM-L6-v2`[18] of the Sentence-BERT model was employed. This model maps sentences and paragraphs to a 384-dimensional dense vector space. Given the large number of comparisons and embeddings required in this project, utilizing a smaller model like `all-MiniLM-L6-v2` allows for significantly faster application performance. While this choice involves a slight trade-off in comparison power compared to larger models, the reduction in accuracy was recognized as negligible for the purposes of this specific application.

The process of calculating the NER-ontology types similarity involves the following steps:

- For each predicted entity, the probability distribution over its predicted NER types is considered.
- For each candidate entity, its associated ontology types are retrieved from the knowledge base.
- The textual definition of each predicted NER type are encoded into embeddings using sBERT model.
- The cosine similarity is computed between the embedding of each predicted NER type and the embedding of each candidate ontology type.
- The final score for a candidate is derived by aggregating these similarities. Different aggregation strategies are explored in the subsequent sub-sections.

Weighted Average of NER Type Embedding Similarities

This scoring strategy calculates the similarity between each predicted NER type for a given entity mention and each of the candidate's ontology types. The cosine similarity scores are then weighted by the probability assigned to each predicted NER type by the NER model. Finally, the average of these weighted similarity scores across all candidate ontology types is taken as the embedding-based score for the candidate.

This method gives more importance to the NER types that the NER model is more confident about. By averaging across the similarities with all the candidate's ontology types, it considers the overall semantic compatibility between the NER predictions and the candidate's semantic profile.

Example presenting how this metric is computed:

Predicted NER types for "Picasso":$\{Person(0.7), Artist(0.2), Painter(0.1)\}$

Candidate ontology types:$\{Artist, Person\}$

[17] `https://www.marqo.ai/course/introduction-to-sentence-transformers`, Accessed: 25.03.2025
[18] `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`

Compute similarities:

$$\text{sim}(\text{Person}, \text{Artist}) = 0.75, \quad \text{sim}(\text{Person}, \text{Person}) = 0.92$$
$$\text{sim}(\text{Artist}, \text{Artist}) = 0.90, \quad \text{sim}(\text{Artist}, \text{Person}) = 0.65$$
$$\text{sim}(\text{Painter}, \text{Artist}) = 0.85, \quad \text{sim}(\text{Painter}, \text{Person}) = 0.70$$

Weight by probabilities and average per ontology type:

$$\text{Artist mean} = \frac{0.7 \cdot 0.75 + 0.2 \cdot 0.90 + 0.1 \cdot 0.85}{1.0} = 0.79$$
$$\text{Person mean} = \frac{0.7 \cdot 0.92 + 0.2 \cdot 0.65 + 0.1 \cdot 0.70}{1.0} = 0.844$$

Final score = average of $\{0.79, 0.844\} \quad 0.817$.

Maximum Similarity with Top-K NER Types

This variant focuses on the top *k* most probable NER types predicted by the NER model for a given entity mention. For each of these top-k types, the cosine similarity is calculated with each of the candidate's ontology types. The highest similarity score obtained between any of the top-k NER type embeddings and any of the candidate's ontology type embeddings is then selected as the score for that candidate.

This approach assumes that even if the NER model produces a distribution over types, the most relevant semantic information might be concentrated in its top few predictions. By taking the maximum similarity, it emphasizes the strongest semantic alignment between the NER predictions and the candidate's types.

Example presenting how this metric is computed:

$$\text{Top-2 NER types:} \{Person(0.7), Artist(0.2)\}$$
$$\text{Ontology types:} \{Artist, Person\}$$

Compute all similarities (as in the first example), then:

$$\max\{\text{sim}(\text{Person}, \text{Artist}), \text{sim}(\text{Artist}, \text{Artist})\} = \max\{0.75, 0.90\} = 0.90 \quad \text{(for Artist)}$$
$$\max\{\text{sim}(\text{Person}, \text{Person}), \text{sim}(\text{Artist}, \text{Person})\} = \max\{0.92, 0.65\} = 0.92 \quad \text{(for Person)}$$

Final score $= \max\{0.90, 0.92\} = 0.92$.

Weighted Sum of Maximum NER Type Embedding Similarities

In this approach, for each predicted NER type and its associated probability, the maximum cosine similarity between its embedding and the embeddings of all the candidate's ontology types is determined. The final score for the candidate is then calculated as the weighted sum of these maximum similarities, where the weights are the probabilities of the respective NER types.

This strategy aims to capture the strongest semantic link between each of the NER model's predictions and the candidate's semantic representation. By summing the weighted maximum similarities, it considers the contribution of each predicted NER type, giving more weight to those with higher probabilities.

Example presenting how this metric is computed:

$$\text{NER types (all 3):}\{Person(0.7), Artist(0.2), Painter(0.1)\}$$
$$\text{Ontology types:}\{Artist, Person\}$$

Compute per-type maxima:

$$\max_{\text{Artist,Person}} \text{sim(Person, } \cdot \text{)} = \max\{0.75,\, 0.92\} = 0.92$$
$$\max_{\text{Artist,Person}} \text{sim(Artist, } \cdot \text{)} = \max\{0.90,\, 0.65\} = 0.90$$
$$\max_{\text{Artist,Person}} \text{sim(Painter, } \cdot \text{)} = \max\{0.85,\, 0.70\} = 0.85$$

Weight and sum:

$$0.7 \cdot 0.92 + 0.2 \cdot 0.90 + 0.1 \cdot 0.85 = 0.644 + 0.18 + 0.085 = 0.909$$

Final score $= 0.909$.

5.3 Selecting the Best Candidate

To effectively use each scoring metric in the final candidate ranking, a machine learning approach was undertaken involving the training of XGBoost classifiers[7] using XGBoost Python package[19]. Recognizing the potential benefit of incorporating embedding-based semantic similarity scores alongside the four contextual features, two distinct sets of XGBoost models were trained and evaluated. The first model focused solely on four contextual scoring metrics, while the second model explored the inclusion of three additional embedding-based metrics that assessed the semantic similarity between the NER-predicted entity types and the ontology classes of the candidate entities.

The XGBoost model was chosen for this task due to its strong predictive performance and its ability to handle non-linear relationships between features. Prior experimentation involved a comparison of XGBoost against other classification algorithms like Random Forest Classifier, Support Vector Machines (SVM), and simple Neural Networks. The evaluation results on a development set indicated that XGBoost consistently achieved the highest accuracy in identifying the correct candidate entity among the retrieved options.

The training data for these models was generated by processing the output of the NER system and the retrieved candidate entities for the mentions in the AIDA training dataset[20]. This process involved

[19] XGBoost Documentation
[20] AIDA train set

the application of each of the previously described scoring metrics to all candidate entities for each mention. The resulting scores, along with a flag indicating whether a candidate matched the gold standard entity URI from the AIDA dataset, were stored in a structured JSON format. This JSON file contained the dataset configuration, the original named entity annotations with their ground truth links, and the predicted candidates for each mention along with their computed scores.

The process of scoring the candidate predictions for each named entity involved the application of each metric. For the first type of XGBoost the first four contextual metrics were calculated. Additionally, for the next model utilizing the NER-Ontology similarity features, the embedding-based similarity scores between NER types and candidate ontology classes were also computed.

The training data for the XGBoost models was prepared by grouping the candidate scores for each entity mention. For each group of candidates, the feature vector consisted of the scores from the selected metrics for all 10 candidates, and the target variable was the index of the candidate that matched the gold standard URI. If the number of candidates returned by DBpedia API was smaller than 10, a padding was added by adding zeros to the input vector.

The XGBoost models were trained using a pipeline that included feature scaling to ensure that metrics with different ranges did not disproportionately influence the learning process. The trained XGBoost classifiers then learned to assign weights to each metric, effectively determining their relative importance in predicting the correct entity link.

A comprehensive feature selection process, using Optuna[2] using the Python Optuna package[21], was undertaken for both sets of models. This involved training and evaluating XGBoost on various combinations of the available metrics. For each combination, the training process was repeated multiple times with different data splits to ensure robust performance evaluation. The performance of each trained model was assessed on the validation set and, crucially, on two independent held-out test datasets: AIDA[22] and ACE2004[23]. The average accuracy across these two test sets served as the primary criterion for selecting the best performing feature combinations.

The best-performing XGBoost models from this feature selection process were saved for integration into the final NED system, allowing for a learned and optimized weighting of the individual scoring metrics.

The detailed outcomes of these experiments are presented in Chapter 7, where full evaluation results are discussed.

[21] `https://github.com/optuna/optuna`
[22] AIDA test set
[23] ACE2004 test set

6. SYSTEM SPECIFICATION

This chapter presents specification of the implemented end-to-end Entity Linking system. An overall architecture is outlined first, followed by detailed descriptions of the two primary modules: Named Entity Recognition and Named Entity Disambiguation, and their subordinate components. The section concludes with an overview of the Graphical User Interface. Components in the system are organized into discrete modules, each encapsulating a specific subset of application logic. This separation of concerns allows easier maintenance, simplifies debugging, and supports easy incremental development of the system. All Python code follows PEP 8 style guide[38], making it standardized, therefore easier to read and understand.

6.1 Entity Linking System

The Entity Linking system is accessed via a web-based GUI implemented in Flask[1]. Each of the two main components: NER and NED components, is further decomposed into specialized modules presented in Figure 6.1, which are described in a greater detail in the following sections.

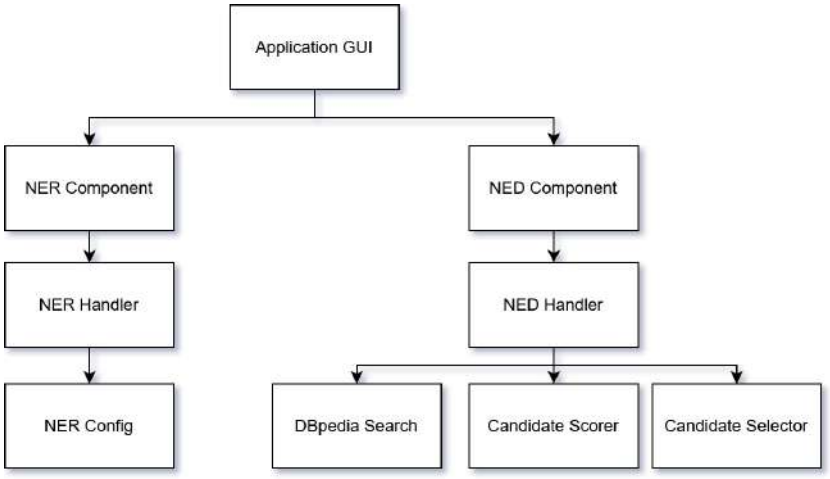

Figure 6.1. Entity Linking System Main Components

6.2 NER Component

The Named Entity Recognition component is responsible for detecting named entities in the input text and assigning them semantic type distributions, which are later used to inform the disambiguation process. This module is designed to support multiple transformer-based SpanMarker models, each trained on different datasets with varying class granularities.

[1]Flask framework documentation

The main NER class, `NERHandler`, is responsible for applying the configured NER model to the input. It tokenizes the text, uses the specified SpanMarker model to detect entities, and converts token-level predictions to character indices for alignment with the original text. Each identified entity is stored along with its most probable type, character span, and a probability distribution over the top three predicted types, which is used during the NED process.

At the core of the `NERHandler` class is the `NERConfig` class, which manages model-specific configurations. It defines a list of the three supported NER models via their names. For each model, class definitions (used for semantic embedding-based comparison of NER and ontology types) are specified, along with a set of entity types that are ignored because they are assumed to be not relevant for linking in this project. After NER finishes, any entity whose top class is on the ignore list is removed from the entities list. The ignored types for each model are as follows:

- `tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context`: none of the defined types were ignored
- `tomaarsen/span-marker-roberta-large-ontonotes5`: `date`, `time`, `percent`, `money`, `quantity`, `ordinal`, `cardinal`, `language`
- `tomaarsen/span-marker-roberta-large-fewnerd-fine-super`: `other-language`

This filtering removes temporal, numeric, and other non-entity mentions from consideration, thereby sharpening focus on linkable entities and improves overall performance.

6.3 NED Component

The Named Entity Disambiguation component is responsible for linking named entities identified by the NER system to their corresponding records in a knowledge base. This is achieved by fetching candidate entities from the knowledge base, calculating multiple relevance scores for each candidate, and selecting the most appropriate candidate using a trained machine learning model.

The central class of this module is the `NEDHandler`, which orchestrates the disambiguation process. It allows to configure the candidate selection strategy. In this project, the knowledge base is restricted to DBpedia, however, adding Wikidata knowledge base would not be a huge effort. The candidate selection model is a trained XGBoost classifier.

The `NEDHandler` iterates over all named entities identified in the input `Text` object. For each entity, it uses the `DBpediaSearch` class to retrieve a list of entity candidates via DBpedia's Lookup API. The DBpedia search implementation is optimized with a persistent, thread-safe disk cache to prevent redundant queries as well as to significantly improve performance of NED process.

Once candidates are retrieved, the `CandidateScorer` class is used to compute a set of relevance scores for each candidate. These include traditional similarity metrics. In addition, if enabled, the scorer integrates embedding-based type similarity scores, comparing NER-predicted types with ontology types associated with each candidate.

After all candidates are scored, the final selection is performed using a trained candidate selector model, implemented in the `CandidateSelector` class. Depending on whether type-based scores are enabled, the system loads either a model based solely on traditional features or one that includes additional embedding-based features. The feature vector is constructed by concatenating the individual scores of the candidates. If the number of candidates returned by the DBpedia API is smaller than the input size expected by the model, the vector is padded accordingly. This ensures compatibility with the fixed-size input required by the XGBoost classifier.

The model outputs the index of the best candidate, which is stored in the `best_candidate_uri` attribute of the `Entity` object. If the prediction is invalid (e.g., index out of range or model failure), the attribute is set to `None`.

6.4 Graphical User Interface

A web-based GUI, implemented in Flask, serves as the primary access point for the Entity Linking system. On a single page, users enter text, choose among pretrained NER models, toggle the inclusion of type-score features, and launch the pipeline. Once processing is complete, results are returned as JSON and rendered interactively. A short demo showing how the system works is available here: demo.

When text is submitted, the selected NER model is applied to detect named entities and compute probability vectors for the top three classes. These probabilities appear as coloured badges, each displaying its confidence to two decimal places. Next, up to ten DBpedia candidates per entity are retrieved and ranked by an XGBoost model; if the type-score option is enabled, embedding-based type similarities are included in the ranking features.

On the frontend, each mention is highlighted as a clickable link. Expanding an accordion panel for a given entity reveals all candidates, with the chosen one marked by a green border and star icon. Candidate cards display the entity label (linked to its URI), ontology type badges, an optional thumbnail image, and a concise table of feature scores. This streamlined interface allows users to inspect NER outputs, compare alternative candidates, and observe score values for each candidate.

Figure 6.2 shows the main interface components, including the input area, NER model selector, and configuration panel, along with the annotated output highlighting the recognized entities and their top predicted classes. The results are shown at the bottom. An example recognized and classified named entity "M&T Bank Stadium" is shown with its top 3 NER types assigned by the `tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context`[2] model.

Figure 6.3 illustrates the expanded candidate view for the entity "AC Milan" after analysing the text using the `tomaarsen/span-marker-roberta-large-fewnerd-fine-super`[3] model. Each candidate displays its DBpedia label, description, ontology types, and detailed feature scores. This allows users to understand the rationale behind the chosen disambiguation.

[2] tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context
[3] tomaarsen/span-marker-roberta-large-fewnerd-fine-super

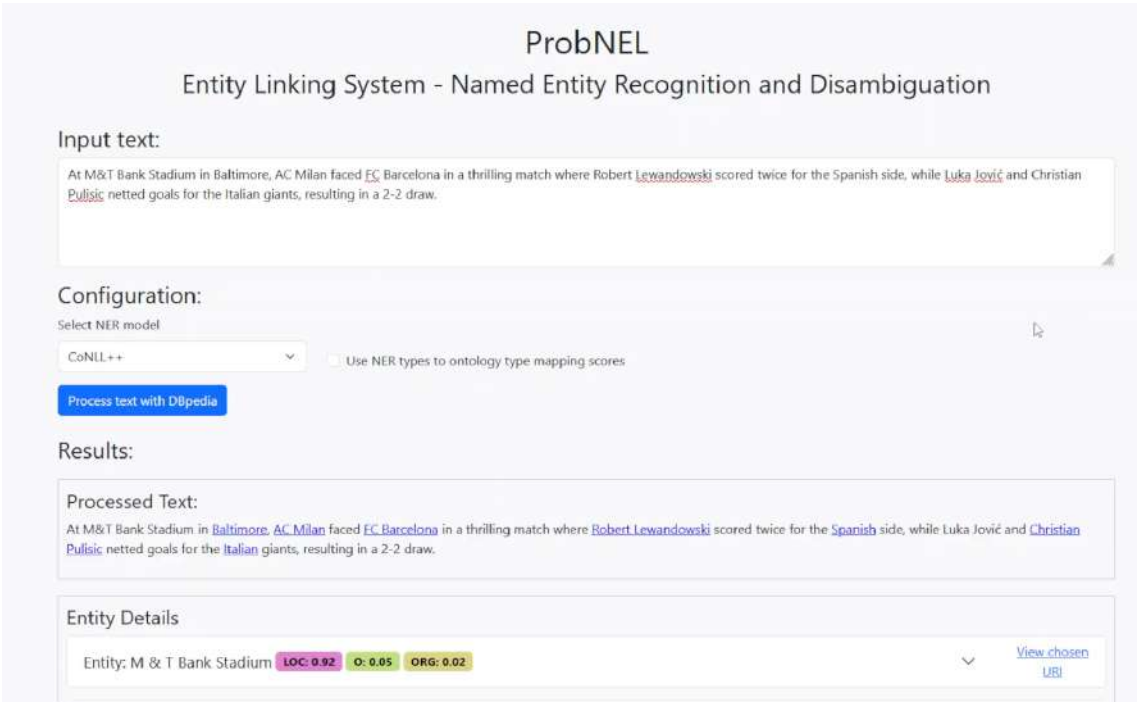

Figure 6.2. Entity Linking System GUI – Input Text, Configuration, and Results

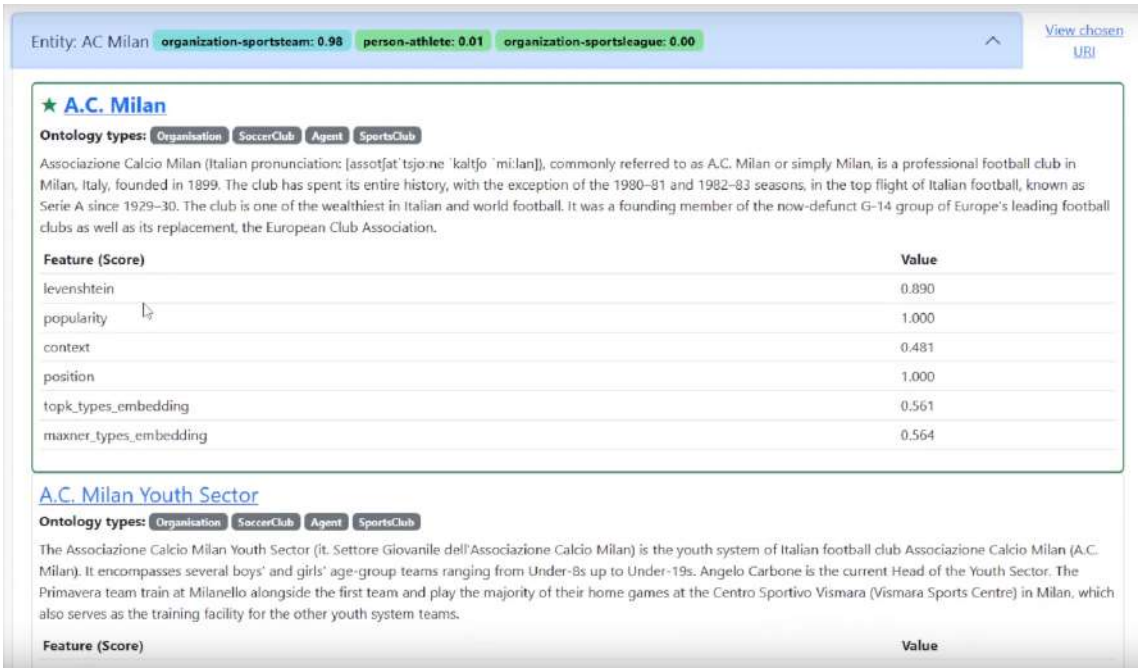

Figure 6.3. Entity Linking System GUI – Candidate List and Feature Scores

7. EXPERIMENTS

This chapter presents a comprehensive evaluation of the developed system including three configurations. The baseline system as well as the second version use golden annotations and focus only on the effectiveness of disambiguation of the candidates fetched from DBpedia. Finally, the third version of the system focuses on the full end-to-end entity linking process. This version of the system firstly performs NER to identify and classify named entities in the input text, and then performs NED to select the best candidate from the DBpedia knowledge base. Each version of the system is assessed in terms of the accuracy of linking using two popular test datasets.

7.1 Evaluation Datasets

For the assessment of Named Entity Disambiguation algorithms, the following datasets were utilised:

- **AIDA-YAGO-CoNLL test dataset**[16]: This dataset is widely used for NED, linking entities from a collection of Wikipedia articles to the YAGO knowledge base. It facilitates the evaluation of disambiguation systems by integrating data from news articles and Wikipedia. This test dataset consists of 230 texts with 4463 ground truth mentions. The original file can be found on GitHub: AIDA test set.
- **ACE2004 test dataset**[28]: Developed by the Linguistic Data Consortium (LDC), this corpus contains text in English, Chinese, and Arabic annotated for entities and relations. It was created for the 2004 Automatic Content Extraction (ACE) technology evaluation, focusing on tasks such as Entity Detection and Recognition and Relation Detection and Recognition across multiple languages and diverse text sources like broadcast news and newswire. This test dataset consists of 119 texts with 257 ground truth mentions. The original file can be found on GitHub: ACE2004 test set.

Both datasets cover a wide range of text genres making them ideal for testing the robustness and adaptability of different disambiguation approaches across different writing styles and domains. By evaluating on these complementary benchmarks, it is ensured that the system not only excels on well-structured encyclopedic text but also generalizes to more varied and noisy real-world data.

7.2 Evaluation Metrics

For evaluating the system, a set of metrics was employed, each tailored to assess specific components of the Entity Linking pipeline. In particular, Recall, Precision, and F1 Score were used to evaluate the Named Entity Recognition step, while Accuracy was employed solely for assessing the linking accuracy in the Named Entity Disambiguation step.

Metrics used for NER evaluation:

- **Precision**: This metric evaluates the quality of the entity predictions made during the NER step. High precision suggests that when the system identifies an entity, it is likely correct.

- **Recall**: This metric measures the system's ability to identify all relevant entities during the NER step. High recall indicates that the system captures most of the entities present in the dataset.

- **F1 Score**: This metric combines Precision and Recall to provide a single measure that balances both concerns. A higher F1 Score reflects a better overall performance of the NER step, particularly when dealing with imbalanced data.

Metric used for NED evaluation:

- **Accuracy**: This metric measures the proportion of correctly linked entities out of the total ground truth entities. A higher accuracy indicates better linking performance.

7.3 Baseline Entity Linking System – Using only Surface Form Matching

The baseline approach provides a simple reference point for later improvements. Named entities are taken directly from the golden annotations in the text. Each entity's surface form is used to query the DBpedia knowledge base via its API. The top result returned by this search is linked to the entity mention. No contextual clues or entity type information is used.

During evaluation, the system processes each annotated entity in the test set by searching DBpedia with its exact surface form. The first record in the search results is assigned to the mention, making the disambiguation decision straightforward and fast.

The performance of this baseline system is summarized in Table 7.1.

Table 7.1. Performance metrics for the baseline system.

Test Dataset	Accuracy
AIDA	0.648
ACE2004	0.720

The observed accuracy was 0.648 on AIDA and 0.720 on ACE2004. These results show that surface-form matching correctly links roughly 65% of mentions in AIDA and 72% in ACE2004. This moderate performance suggests that many entities possess distinctive names, but the remaining error rate underscores the limitations of string-only matching and motivates the integration of contextual or entity-type information to improve disambiguation accuracy.

7.3.1 Candidate Rank Position Analysis

Two plots showing the distribution of correct candidate positions are presented in Figures 7.1 and 7.2.

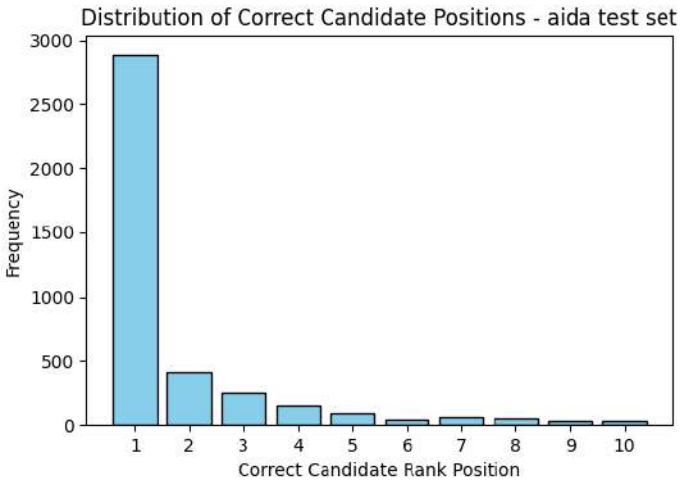

Figure 7.1. Distribution of Correct Candidate Positions - AIDA test set

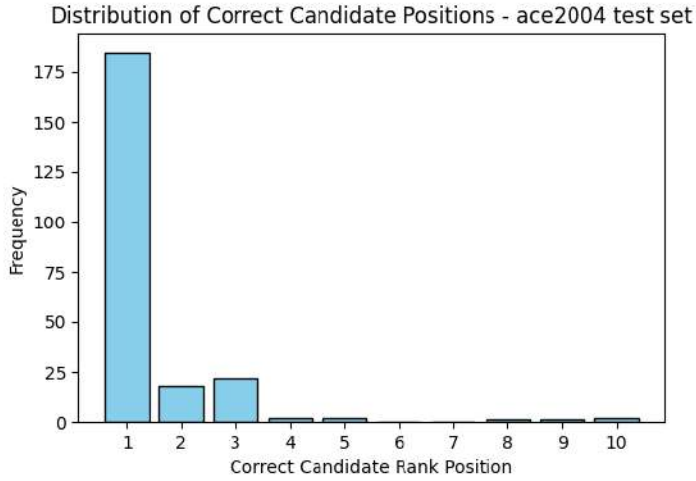

Figure 7.2. Distribution of Correct Candidate Positions - ACE2004 test set

Figures 7.1 and 7.2 illustrate how often the correct entity appears at each rank in the top-10 candidate lists generated by the simple surface form matching approach. In both datasets, most correct entities are found at rank 1. However, the long tail observed in both figures indicates that in a non-negligible number of cases, the correct entity was positioned further down the list.

Next, cumulative frequency plots in Figures 7.3 and 7.4 show the percentage of correct candidates retrieved within the top-k ranks. In the AIDA set, about 73% are at rank 1 and 90% are within the top 5, while in the ACE2004 set, approximately 80% of correct entities are at rank 1 and more than 95'% are within the top 3.

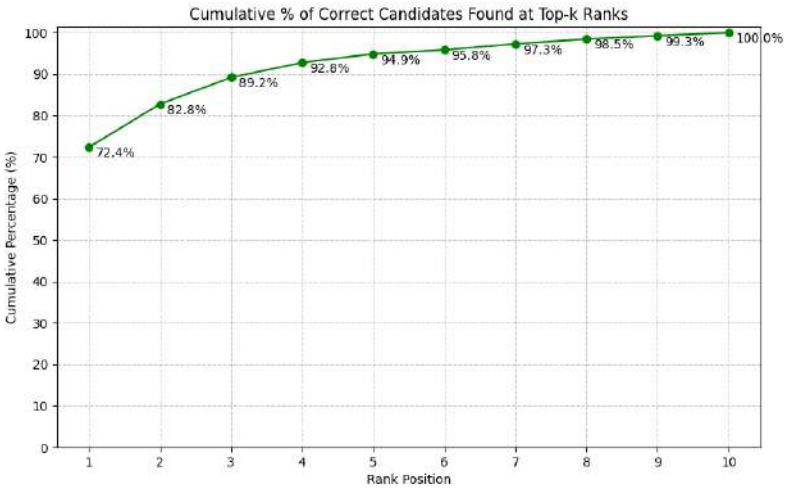

Figure 7.3. Distribution of Correct Candidate Positions - AIDA test set

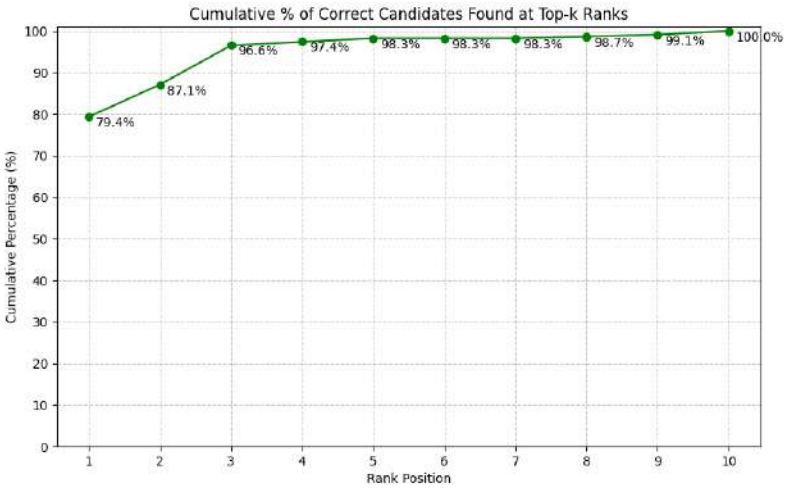

Figure 7.4. Distribution of Correct Candidate Positions - ACE2004 test set

These results demonstrate the effectiveness of a top-10 candidate list. In case of the AIDA test set: 72.38% of correct entities are found at rank 1, 94.89% at the top 5, and 100% by rank 10. While in case of ACE2004 test set: 79.40% at rank 1, 98.28% by the top 5, and 100% by rank 10.

In case of these two test sets, by retrieving the top ten candidates, more than 95% of correct matches appear within the first five positions, and every correct entity is included in the candidates list. This insight suggests that, in real-world scenarios the system will have enough candidates to select the correct one.

7.4 Entity Linking System Using Only Contextual Features

In this version, the simple surface-form matching baseline is enhanced by incorporating four basic scoring metrics into an XGBoost classifier. First, candidate entities are retrieved from DBpedia using the exact surface form of each mention (as in the baseline). Then, for each candidate, the following scores are computed:

- **Levenshtein Distance** – measures string similarity between the mention and the candidate label
- **Popularity** – reflects how frequently the candidate appears in DBpedia
- **Context Similarity** – captures how well the surrounding text of the named entity matches the candidate's DBpedia description
- **Position Score** – encodes the candidate's rank in the DBpedia search results

An XGBoost classifier was selected after its superior performance was demonstrated over other models during feature-selection experiments (Chapter 5.3). During both training and testing, golden annotations were used, ensuring that every named entity was included in the analysis. This allowed for a focus on the analysis of the disambiguation power of this approach, and the use of any NER model was made unnecessary. Furthermore, the fact that the NER types of named entities were not utilized in this approach was also a reason for using the golden annotations instead of a NER model.

The overall workflow consisted the following steps:

1. **Candidate Retrieval and Scoring:** For each named entity mention, up to ten candidates were fetched from DBpedia. Each candidate was scored using the four contextual metrics.
2. **Feature-Selection Loop:** A series of experiments was run in which the XGBoost model was trained on the AIDA training set using different combinations of the four metrics.
3. **Cross-Dataset Evaluation:** After each training run, evaluation was performed on the AIDA and ACE2004 test sets using their gold-standard named entity annotations.
4. **Best Feature Set Identification:** The combination of metrics that produced the highest average test accuracy across AIDA and ACE2004 was chosen.

Using this process, it was found that including all four contextual features yields the best results. The final XGBoost model achieved an average test accuracy of 0.853 on the AIDA and ACE2004 test datasets. Detailed performance metrics are shown in Table 7.2.

Table 7.2. Performance metrics for the enhanced system.

Test Dataset	Accuracy
AIDA	0.865
ACE2004	0.841

The observed accuracy was 0.865 on AIDA and 0.841 on ACE2004. This marked improvement over the surface-form baseline demonstrates that combining Levenshtein distance, popularity, context similarity, and position scores in an XGBoost classifier substantially enhances disambiguation accuracy. The results present how multiple complementary features jointly contribute to more accurate and reliable entity linking.

7.5 Entity Linking System Using Contextual and Additional Embedding-based Features

The final version of the system adds three embedding-based scores on top of the four contextual metrics. In this full pipeline, each mention is first recognised by a NER model, and then up to ten candidates are retrieved from DBpedia. Every candidate is scored using:

- **Levenshtein Distance**
- **Popularity**
- **Context Similarity**
- **Position Score**
- **Weighted Average of NER Type Embedding Similarities**
- **Maximum Similarity with Top-K NER Types**
- **Weighted Sum of Maximum NER Type Embedding Similarities**

The training data is taken from the AIDA training set, which originally contains 18 395 ground-truth mentions across 940 texts. For the needs of this project, only those named entities that both appear in the gold annotations and are correctly recognised by the NER model were used. In this way, each entity is provided with a set of NER types predicted by the model (used during candidate disambiguation) and a target URI from the gold annotation (used to evaluate linking).

Each NER model predicted named entities with a different recall and precision, as presented in Table 7.3.

Table 7.3. Predictions of Different NER Models on the AIDA Train Set

NER Model	Recall	Precision	F1 Score
Model trained on the CoNLL++ dataset	0.855	0.041	0.077
Model trained on the OntoNotes 5.0 dataset	0.607	0.038	0.071
Model trained on the Few-NERD dataset	0.580	0.038	0.070

A clear pattern can be see in the collected statistics: all models tend to identify substantially more spans as entities than those actually present in the gold annotations of AIDA train set. For example, while the CoNLL++ model attains the highest recall (0.855), its precision remains extremely low (0.041), indicating that many detected spans do not correspond to true entities. Similarly, the

OntoNotes and Few-NERD models record precision values of just 0.038 despite moderate recall (0.607 and 0.580 respectively), confirming that they too over-generate entity predictions.

Based on these NER model predictions, each model contributed a subset to the final training dataset, which comprised 37 629 examples in total. Since some entities may have been detected by more than one model, duplicates can appear in the combined set - however, their assigned classes will differ because each model uses a distinct label scheme. The contributions of each of the NER models are:

- 41.91% from dataset with predictions made by the model trained on the CoNLL++ dataset
- 29.67% from dataset with predictions made by the model trained on the OntoNotes 5.0 dataset
- 28.42% from dataset with predictions made by the model trained on the Few-NERD dataset

Moreover, it is important to point out that many of the candidates fetched via the DBpedia API lacked ontology type labels. In the adjusted AIDA training set, 29.8% of candidates have no ontology type defined (see Figure 7.5). Therefore, values of the new scores was automatically set to 0 for them, since the NER types of named entity could not be compared with the ontology types of candidates. In the list of 10 candidates, there were usually 1 to 3 candidates without the ontology types defined.

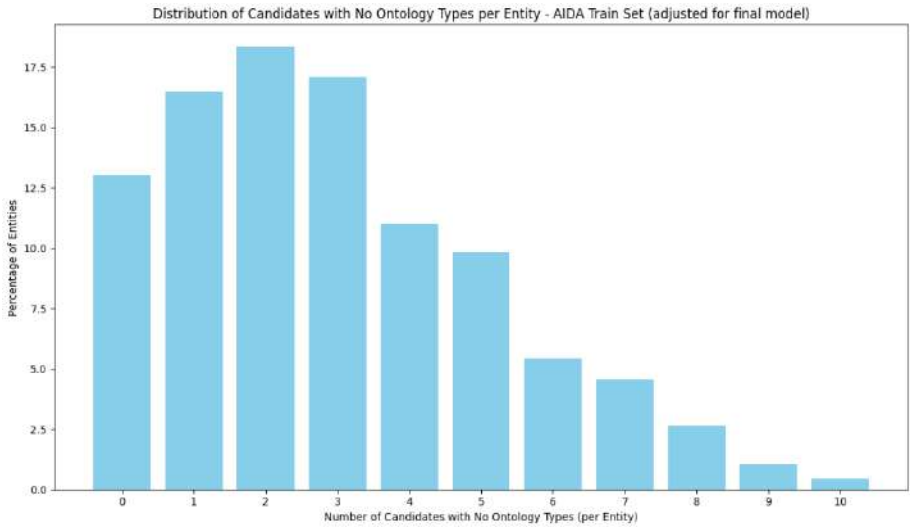

Figure 7.5. Distribution of Candidates with No Ontology Types per Entity - AIDA Train Set (adjusted for training an XGBoost classifier used in the final version of the system)

To further improve the Entity Linking system, an additional XGBoost model was trained using an adjusted and extended version of the AIDA training dataset. The process for feature selection mirrored that employed for the system relying solely on contextual features. The feature selection process identified, that the best feature combination would be to use all of the baseline features as well as two additional features, namely: Maximum Similarity with Top-K NER Types and Weighted Sum of Maximum NER Type Embedding Similarities.

A key advantage of using this modified training data is the straightforward integration of new NER models into the system. In contrast to methods that necessitate the training of a specific XGBoost model for each NER model, the current XGBoost model demonstrates a significant ability to work with

various models without the need of any additional adjustments in the workflow. The trained XGBoost model performs effectively across different NER outputs without the need for additional training.

The end-to-end system was evaluated on the original AIDA and ACE2004 test sets. As in training, only those mentions that were both present in the gold annotations and correctly recognised by each NER model were included in the evaluation.

Each NER model achieved different recall rates on the AIDA and ACE2004 test sets, as shown in Tables 7.4 and 7.5.

Table 7.4. Predictions of Different NER Models on the AIDA Test Set

NER Model	Recall	Precision	F1 Score
Model trained on the CoNLL++ dataset	0.882	0.042	0.080
Model trained on the OntoNotes 5.0 dataset	0.613	0.039	0.074
Model trained on the Few-NERD dataset	0.577	0.039	0.073

Table 7.5. Predictions of Different NER Models on the ACE2004 Test Set

NER Model	Recall	Precision	F1 Score
Model trained on the CoNLL++ dataset	0.852	0.100	0.179
Model trained on the OntoNotes 5.0 dataset	0.770	0.089	0.160
Model trained on the Few-NERD dataset	0.716	0.088	0.156

Overall, both tables confirm that all three NER models detect many more spans than actually exist in the gold annotations, yielding high recall but very low precision. Presented results underline that, while CoNLL++ finds the most true mentions, all models generate a large volume of false positives.

This way 6 different test datasets were created. The evaluation results using these datasets are presented in the Table7.6 and Table7.7.

Table 7.6. Performance metrics for the final system with different NER models on the AIDA dataset.

EL System Using NER Model	Accuracy
CoNLL++ Model	0.873
OntoNotes Model	0.861
Few-NERD Model	0.873

Table 7.7. Performance metrics for the final system with different NER models on the ACE2004 dataset.

EL System Using NER Model	Accuracy
CoNLL++ Model	0.863
OntoNotes Model	0.864
Few-NERD Model	0.904

On the AIDA dataset, similar patterns were observed across all three configurations - each using results from a different NER model. Each system achieved high accuracy rates, with values close to or above 0.86. This outcome suggests that the inclusion of the new type-based features allowed the Entity Linking system to perform effectively.

A similar trend was observed on the ACE2004 dataset. The overall accuracy ranged from 0.863 to 0.904, with the Few-NERD-based system achieving the highest value. This suggests that, while different NER models bring unique type information and coverage, all can contribute positively to the final linking step when type-based features are employed.

Overall, the results suggest that the integration of features derived from NER models can positively influence the end-to-end linking accuracy. These final results indicate that incorporating NER-based type similarity features can enhance the robustness of the EL system. Even using just four basic types can lead to improvements, while in some cases more detailed type information from the NER model can further enhance disambiguation performance, as seen in case ACE2004 test dataset. It is clear that the overall effectiveness of a two-step entity linking system is limited by the first step: if the NER model poorly identifies named entities (low recall), the disambiguation step cannot compensate, regardless of how well it is designed and how many additional features it uses in the disambiguation process.

7.5.1 Evaluation Challenges

Evaluating an end-to-end entity linking system presents several challenges. Widely used datasets referenced in numerous publications lack a standardised method for annotating named entities, leading to inconsistencies in entity definitions. Moreover, the criteria for determining a correct entity link can be ambiguous.

For example, a named entity with the surface form *"Tom Waits"* may be linked by the system to `http://dbpedia.org/resource/PEHDTSCKJBMA`, whereas the reference dataset provides `http://dbpedia.org/resource/Tom_Waits`. At first glance, this may appear incorrect, yet the former link redirects to the latter, raising questions about annotation validity.

Additionally, despite efforts to follow academic standards, research groups frequently develop different annotation approaches for entity linking benchmarks[22]. These methodological differences create challenges, for example: one entity linking benchmark requires to link *Biden* to Joe Biden DBpedia article, while another requires the full name *Joe Biden"* to be linked to the same article. In this case, both approaches might technically follow their respective guidelines, but when evaluating an end-to-end entity linking system both answers should be correct when looking from the real-life perspective, where the systems can have multiple valid annotation strategies rather than a single correct answer. Fortunately, emerging solutions address this through flexible benchmarking, by allowing multiple different annotations, particularly in ambiguous cases where entity boundaries or naming conventions might be unclear[5]. This approach better reflects real-world language complexity while maintaining evaluation rigour.

8. FUTURE WORKS

After a retrospective and deep analysis of the results, the list of possible future improvements and extensions of the system was created:

- **Handling Missing Ontology Types** - Approximately 30% of DBpedia candidates fetched for named entities in the AIDA training set lack ontology-type labels. An improvement that might solve this problem is to use the DBpedia category tags or related RDF properties when no "rdf:type" is defined. This should improve type-based scoring for those candidates and possibly improve the accuracy of the end-to-end entity linking process.
- **New Techniques for NER-Ontology Similarities Scores** - Alternative methods for comparing NER-type and ontology-type embeddings could be explored. For instance, Euclidean distance or other metrics might be used instead of cosine similarity. Such new features may capture different aspects of type relationships and improve disambiguation accuracy.
- **Enriching the Training Corpus** - Training data might be extended with additional sources, for example by adding data from other datasets. More varied examples should help the XGBoost model learn stronger feature weights and improve generalisation.
- **Multilingual and Domain-Specific Evaluation** - To better test real-world robustness, the system might be tested on non-English datasets (for example, ACE Chinese and Arabic) and on some specialised domains (such as biomedical or legal texts). Adapting the disambiguation strategy on these domains may reveal new challenges and opportunities and reduce the system's generalisation error.
- **Interactive GUI Enhancements** - In the current web interface, the user can inspect candidate details and individual feature scores. In the future a visualisation widget that displays a small knowledge graph of linked entities and their relations might be added. This would make the system even more transparent and help users understand the semantic context of entities and candidates as well as explore connections between identified named entities.

9. CONCLUSION

This thesis presents a practical, end-to-end Entity Linking system that combines Named Entity Recognition with context-aware Named Entity Disambiguation over the DBpedia knowledge base. Three variants of entity linking approach were developed and evaluated, namely:

- A baseline simple surface-form matching method linked every mention by its exact spelling. In case of this system golden annotations from the test dataset were used. The accuracy of linking remained moderate and highlighted the need for feature engineering and context understanding features.
- An enhanced version of the system uses a model that includes four contextual disambiguation features, which are used as a representation of each single candidate. These features are then used as input to an XGBoost classifier, which chooses the most appropriate candidate. This approach was also tested using golden annotations. This strategy increased linking accuracy on both benchmarks.
- The third version of the system was designed as a full end-to-end pipeline that adds three embedding-based NER type-similarity scores on top of the contextual features. By leveraging NER-type distributions over fine-grained classes from one of three NER models, this version achieved very high linking accuracy on AIDA and ACE2004 test datasets, demonstrating the benefit of incorporating additional features measuring the similarity between the NER types of an identified mention and the ontology types of candidates.

Overall, the results confirm that a simple training setup combined with a learned ranking model can yield very satisfying performance without requiring massive annotated corpora, as well as training time and computational resources. The system is relatively fast, highly interpretable, and adaptable: different NER models and new domains can be plugged in with minimal adjustments thanks to its modularity and already trained candidate selection models.

This framework creates a solid foundation for lightweight yet accurate Entity Linking systems, which are based on both contextual features as well as features that leverage named entity type information assigned during the Named Entity Recognition phase. This approach allowed to enhance the Named Entity Disambiguation process and achieve higher linking accuracy, than while using solely contextual information.

Bibliography

[1] Tom Aarsen et al. *Spanmarker for named entity recognition*. 2023.

[2] Takuya Akiba et al. "Optuna: A next-generation hyperparameter optimization framework". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2623–2631.

[3] Sören Auer et al. "Dbpedia: A nucleus for a web of open data". In: *international semantic web conference*. Springer. 2007, pp. 722–735.

[4] Tom Ayoola et al. "Refined: An efficient zero-shot-capable approach to end-to-end entity linking". In: *arXiv preprint arXiv:2207.04108* (2022).

[5] Hannah Bast, Matthias Hertel, and Natalie Prange. "A Fair and In-Depth Evaluation of Existing End-to-End Entity Linking Systems". In: *arXiv preprint arXiv:2305.14937* (2023).

[6] Youcef Benkhedda et al. "Enriching the metadata of community-generated digital content through entity linking: An evaluative comparison of state-of-the-art models". In: *Proceedings of the 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature (LaTeCH-CLfL 2024)*. 2024, pp. 213–220.

[7] Tianqi Chen and Carlos Guestrin. "Xgboost: A scalable tree boosting system". In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, pp. 785–794.

[8] Tao Cheng, Xifeng Yan, and Kevin Chen-Chuan Chang. "Entityrank: Searching entities directly and holistically". In: *Proceedings of the 33rd international conference on Very large data bases*. 2007, pp. 387–398.

[9] Alexis Conneau et al. "Unsupervised cross-lingual representation learning at scale". In: *arXiv preprint arXiv:1911.02116* (2019).

[10] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[11] Ning Ding et al. "Few-nerd: A few-shot named entity recognition dataset". In: *arXiv preprint arXiv:2105.07464* (2021).

[12] Tome Eftimov, Barbara Koroušić Seljak, and Peter Korošec. "A rule-based named-entity recognition method for knowledge extraction of evidence-based dietary recommendations". In: *PloS one* 12.6 (2017), e0179488.

[13] Zheng Fang et al. "Joint entity linking with deep reinforcement learning". In: *The world wide web conference*. 2019, pp. 438–447.

[14] Octavian-Eugen Ganea and Thomas Hofmann. "Deep joint entity disambiguation with local neural attention". In: *arXiv preprint arXiv:1704.04920* (2017).

[15] Ralph Grishman and Beth M Sundheim. "Message understanding conference-6: A brief history". In: *COLING 1996 volume 1: The 16th international conference on computational linguistics*. 1996.

[16] Johannes Hoffart et al. "Robust Disambiguation of Named Entities in Text". In: *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*. Ed. by Regina Barzilay and Mark Johnson. Edinburgh, Scotland, UK.: Association for Computational Linguistics, July 2011, pp. 782–792. URL: https://aclanthology.org/D11-1072.

[17] Johannes Hoffart et al. "Robust disambiguation of named entities in text". In: *Proceedings of the 2011 conference on empirical methods in natural language processing*. 2011, pp. 782–792.

[18] Feng Hou et al. "Improving entity linking through semantic reinforced entity embeddings". In: *arXiv preprint arXiv:2106.08495* (2021).

[19] Zhiheng Huang, Wei Xu, and Kai Yu. "Bidirectional LSTM-CRF models for sequence tagging". In: *arXiv preprint arXiv:1508.01991* (2015).

[20] Heng Ji and Ralph Grishman. "Knowledge base population: Successful approaches and challenges". In: *Proceedings of the 49th annual meeting of the association for computational linguistics: Human language technologies*. 2011, pp. 1148–1158.

[21] Yankai Lin et al. "Neural relation extraction with selective attention over instances". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2016, pp. 2124–2133.

[22] Xiao Ling, Sameer Singh, and Daniel S Weld. "Design challenges for entity linking". In: *Transactions of the Association for Computational Linguistics* 3 (2015), pp. 315–328.

[23] Xiao Ling and Daniel Weld. "Fine-grained entity recognition". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 26. 1. 2012, pp. 94–100.

[24] Yinhan Liu. "Roberta: A robustly optimized bert pretraining approach". In: *arXiv preprint arXiv: 1907.11692* 364 (2019).

[25] Mauricio Marrone, Sascha Lemke, and Lutz M Kolbe. "Entity linking systems for literature reviews". In: *Scientometrics* 127.7 (2022), pp. 3857–3878.

[26] Pablo N Mendes et al. "DBpedia spotlight: shedding light on the web of documents". In: *Proceedings of the 7th international conference on semantic systems*. 2011, pp. 1–8.

[27] Matthew Michelson and Sofus A Macskassy. "Discovering users' topics of interest on twitter: a first look". In: *Proceedings of the fourth workshop on Analytics for noisy unstructured text data*. 2010, pp. 73–80.

[28] Alexis Mitchell et al. *ACE 2004 Multilingual Training Corpus*. Web Download. LDC Catalog No.: LDC2005T09. Philadelphia, 2005. URL: https://catalog.ldc.upenn.edu/LDC2005T09.

[29] Nita Patil, Ajay Patil, and BV Pawar. "Named entity recognition using conditional random fields". In: *Procedia Computer Science* 167 (2020), pp. 1181–1188.

[30] Jonathan Raiman and Olivier Raiman. "Deeptype: multilingual entity linking by neural type system evolution". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018.

[31] Manoj Prabhakar Kannan Ravi et al. "CHOLAN: A modular approach for neural entity linking on Wikipedia and Wikidata". In: *arXiv preprint arXiv:2101.09969* (2021).

[32] Nils Reimers and Iryna Gurevych. "Sentence-bert: Sentence embeddings using siamese bert-networks". In: *arXiv preprint arXiv:1908.10084* (2019).

[33] Erik F Sang and Fien De Meulder. "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition". In: *arXiv preprint cs/0306050* (2003).

[34] Wei Shen, Jianyong Wang, and Jiawei Han. "Entity linking with a knowledge base: Issues, techniques, and solutions". In: *IEEE Transactions on Knowledge and Data Engineering* 27.2 (2014), pp. 443–460.

[35] Sonit Singh. "Natural language processing for information extraction". In: *arXiv preprint arXiv: 1807.02383* (2018).

[36] Julian Szymański et al. "Entity Annotation with Wikipedia Using Neural Networks". In: *International Conference on Computer Information Systems and Industrial Management*. Springer. 2024, pp. 272–284.

[37] Simone Tedeschi et al. "Named Entity Recognition for Entity Linking: What Works and What's Next". In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. Ed. by Marie-Francine Moens et al. Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 2584–2596. DOI: 10.18653/v1/2021.findings-emnlp.220. URL: https://aclanthology.org/2021.findings-emnlp.220.

[38] Guido Van Rossum, Barry Warsaw, and Nick Coghlan. "PEP 8–style guide for python code". In: *Python. org* 1565 (2001), p. 28.

[39] Daniel Vollmers et al. "Contextual Augmentation for Entity Linking using Large Language Models". In: *Proceedings of the 31st International Conference on Computational Linguistics*. 2025, pp. 8535–8545.

[40] Denny Vrandečić and Markus Krötzsch. "Wikidata: a free collaborative knowledgebase". In: *Communications of the ACM* 57.10 (2014), pp. 78–85.

[41] Ralph Weischedel et al. "OntoNotes Release 5.0 LDC2013T19". In: (2013). URL: https://catalog.ldc.upenn.edu/LDC2013T19.

[42] Ledell Wu et al. "Scalable zero-shot entity linking with dense entity retrieval". In: *arXiv preprint arXiv:1911.03814* (2019).

[43] Wenzheng Zhang, Wenyue Hua, and Karl Stratos. "Entqa: Entity linking as question answering". In: *arXiv preprint arXiv:2110.02369* (2021).

[44] Yuanzhe Zhang et al. "A joint model for question answering over multiple knowledge bases". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. 2016.

LIST OF FIGURES

LIST OF TABLES

LIST OF LISTINGS

APPENDIX WITH NER TYPES DEFINITIONS

This appendix provides the class labels and their definitions for each of the three NER models used in the Entity Linking system. Each model's label set is presented in its own subsection.

Model: `tomaarsen/span-marker-xlm-roberta-large-conllpp-doc-context`

PER Proper names of individuals, including first names, last names, fictional names, and unique nicknames.

LOC Names of geographical locations such as cities, countries, states, provinces, and other physical locations.

ORG Names of organizations including companies, government agencies, political parties, educational institutions, and sports teams.

MISC Other named entities that do not fit into PER, LOC, or ORG categories, such as nationalities, artistic works, events, and other miscellaneous proper nouns.

Model: `tomaarsen/span-marker-roberta-large-ontonotes5`

PERSON Proper names of people including first names, last names, individual or family names, fictional names and unique nicknames.

NORP Adjectival forms of GPE and non-GPE place names (such as American), named religions, heritage, and political affiliation.

FAC Names of man-made structures, including buildings, airports, stations, infrastructures (bridges and streets), monuments, oil fields, golf courses, hospitals, zoos, shopping centres, etc.

ORG Names of companies, government agencies, political parties, educational institutions, sports teams, hospitals, museums, libraries, etc.

GPE Names of geographical administrative entities including countries, villages, cities, states, provinces, prefectures, and other forms of municipalities.

LOC Names of locations other than GPEs including celestial bodies, stars, continents, mountains, oceans, coasts, rivers, lakes, borders, etc.

PRODUCT Names of any product including non-commercial vehicles (automobiles, rockets, aircraft, ships).

EVENT Named events and phenomena including natural disasters, hurricanes, revolutions, battles, wars, demonstrations, concerts, sports events, etc.

WORK_OF_ART Titles of books, songs, films, plays and other creations such as awards, stock price indexes, and social security systems including health insurance systems or pension plans.

LAW Named legal documents including laws, treaties, sections, and chapters.

LANGUAGE Any named language including programming languages.

Model: `tomaarsen/span-marker-roberta-large-fewnerd-fine-super`

art-broadcastprogram Broadcast programmes including TV and radio shows.

art-film Film titles and cinematic works.

art-music Music-related works including performances, bands, and symphonies.

art-other Other artistic creations not covered by film, music, painting, or written art.

art-painting Titles of paintings and art reproductions.

art-writtenart Literary and written art forms, including books and scripts.

building-airport Names of airports and aviation terminals.

building-hospital Hospitals and medical facilities.

building-hotel Hotels and lodging establishments.

building-library Libraries and book repositories.

building-other Other types of buildings not categorised elsewhere.

building-restaurant Restaurants and dining establishments.

building-sportsfacility Sports facilities and arenas.

building-theater Theatres and performance venues.

event-attack/battle/war/militaryconflict Military conflicts including attacks, battles, wars, and military engagements.

event-disaster Natural or man-made disasters and catastrophic events.

event-election Elections and voting events.

event-other Other events not classified elsewhere.

event-protest Protests and demonstrations.

event-sportsevent Sports events and competitions.

location-GPE Geopolitical entities, typically countries, states, or cities.

location-bodiesofwater Significant bodies of water such as lakes, rivers, and coasts.

location-island Names of islands and archipelagos.

location-mountain Mountain ranges, peaks, and related geographical features.

location-other Other location names not covered by standard geographical categories.

location-park Parks and recreational areas.

location-road/railway/highway/transit Transportation-related locations such as roads, railways, highways, and transit systems.

organization-company Companies and corporate entities.

organization-education Educational institutions and organisations.

organization-government/governmentagency Government bodies and agencies.

organization-media/newspaper Media organisations and newspaper titles.

organization-other Other organisational entities not covered by other categories.

organization-politicalparty Political parties and related organisations.

organization-religion Religious organisations and groups.

organization-showorganization Organisations related to shows, entertainment, and performance groups.

organization-sportsleague Sports leagues and associations.

organization-sportsteam Sports teams and clubs.

other-astronomything Celestial objects and astronomy-related terms.

other-award Awards and honours.

other-biologything Biological terms and entities.

other-chemicalthing Chemical substances and compounds.

other-currency Currency symbols or names.

other-disease Diseases and medical conditions.

other-educationaldegree Academic degrees and certifications.

other-god Deities and divine entities.

other-law Legal terms and law-related entities.

other-livingthing Living organisms and fauna.

other-medical Medical specialties, professionals, or terms.

person-actor Actors and performers in film, television, or theatre.

person-artist/author Artists and authors.

person-athlete Athletes and sports figures.

person-director Film or stage directors.

person-other Other persons not categorised elsewhere.

person-politician Politicians and political figures.

person-scholar Scholars, researchers, and academics.

person-soldier Military personnel and soldiers.

product-airplane Aircraft and aeroplanes.

product-car Cars and automobiles.

product-food Food items or brands.

product-game Games or video game titles.

product-other Other products or merchandise.

product-ship Ships and maritime vessels.

product-software Software products or applications.

product-train Trains and railway vehicles.

product-weapon Weapons and armaments.